Summary of "Agents as a New System Workload"

Shreya Nadiger (snadiger), Janani Giridharan (jananig),
Bhargav Keralapur Srinidhi (bsrinid), Haripreeth Dwarakanath Avarur (havarur)

Section 1: Research Direction of the Field

Paper 1.q1. : What is the research problem, and why is it important?

This paper aims to explore the system level infrastructure of "dynamic reasoning" in LLM based AI agents especially focusing on how the agent design and test-time scaling affect latency, GPU resource utilisation, energy consumption as well as datacenter wise scale power requirements. Reasoning strategies and agent architectures are designed with infrastructure cost and scalability in mind, the authors explicitly state that they aim to provide a "rigorous, quantitative evaluation of the computational and infrastructural costs of dynamic reasoning" and systematically characterize:- resource utilization, latency, energy demands and infrastructure implications of AI agents.

Paper 1.q2.: What was the state of related work before the lecture papers appeared?

Before this work, first, test-time scaling increased LLM reasoning computation through methods such as Chain-of-Thought and Tree-of-Thought. Second, LLM-based agents extended this idea by introducing multi-step reasoning, tool use, planning, reflection, and interaction with external environments. The authors explain how CoT and Tree-of-Thought increased computation during inference and how agents extended test-time scaling into dynamic, tool-interacting workflows.

Paper 1.q3.: At a high level, what do the lecture papers solve, and where do they stand in the area?

The paper evaluates representative agents including CoT, ReAct, Reflexion, LATS, and LLMCompiler across several workloads, examining their LLM/tool invocation patterns, latency, resource utilization, energy consumption, and power requirements, studies how agent design choices and test-time scaling affect the accuracy–cost trade-off. Previous agent research focused largely on task success and reasoning behavior, while the deployment costs and system/infrastructure implications were insufficiently studied. It provides a systems/infrastructure perspective on agentic AI, showing that increasing reasoning computation can also produce substantial and sometimes diminishing infrastructure costs.

Paper 2.q1.: What is the research problem, and why is it important?

This paper aims to explore the system-level execution and infrastructure challenges of Agentic AI, with a particular focus on understanding how CPU-side tool execution interacts with GPU-based LLM inference and how these interactions affect end-to-end latency, throughput, energy consumption, and overall resource utilisation. The broader research direction is towards building efficient and scalable Agentic AI serving systems, where both CPU and GPU resources are jointly considered rather than optimising the LLM/GPU component in isolation.

Paper 2.q2.: What was the state of related work before the lecture papers appeared?

Before this work, Agentic AI research had largely been characterized from the perspective of agent capabilities, such as what tools an agent can use, how it reasons, and what tasks it can accomplish, while AI-systems research had predominantly focused on improving the efficiency of conventional LLM inference, particularly through GPU optimisation and KV-cache management.

Paper 1.q3.: At a high level, what do the lecture papers solve, and where do they stand in the area?

In Sections 2 and 3,the authors establish their workload taxonomy and perform runtime characterization across representative Agentic AI workloads, examining metrics such as end-to-end latency, throughput, CPU/GPU utilisation, and energy consumption and then move beyond characterization to optimisation: in Section 4, they introduce COMB (CPU-Aware Overlapped Micro-Batching) to better coordinate CPU and GPU execution for homogeneous workloads, while Section 5 introduces MAS (Mixed Agentic Scheduling) to handle heterogeneous workloads containing different CPU- and GPU-resource requirements.

Section 2: Lecture Papers

Paper 1: The Cost of Dynamic Reasoning

- **Problem and contribution:** Agent workflows take different paths as they run, making latency, memory demand, and serving capacity difficult to predict. Infrastructure costs across agent designs remain insufficiently understood. The authors measure these costs and the effects of test-time scaling. They find diminishing accuracy gains from additional reasoning and benefits from prefix caching. Sequential reasoning limits simultaneous resource demand but increases latency, while parallel exploration can improve accuracy and response time at the cost of more memory.
- **Evaluation:** Using Llama-3.1 models on A100 GPUs, the authors evaluate individual requests and concurrent serving across four task categories. Prefix caching improves throughput by an average of 5.62× in the tested ReAct serving workloads. Comparisons between different agent and chatbot tasks do not isolate overhead at comparable answer quality.
- **Shortcomings:** Fifty questions per setting and one model family limit generalization. Energy projections omit batching, potentially overstating GPU energy, and CPU, networking, and cooling, understating total energy. These unquantified opposing effects make deployment estimates uncertain. The batching omission does not apply to the concurrent-serving experiments.

Paper 2: A CPU-Centric Perspective on Agentic AI Execution

- **Problem and contribution:** Insufficient coordination between CPU tools and GPU inference causes contention that separate component optimizations may leave unresolved. The authors characterize workloads and propose two scheduling methods. COMB splits and overlaps batches or limits concurrency under continuous arrivals. MAS separates CPU-heavy and GPU-heavy admissions with shared overflow slots. Standalone overlap can improve median latency while worsening tail latency; suitable concurrency limits under continuous arrivals can improve both.
- **Evaluation:** The authors characterize five workloads on two platforms. In one System 2 web-agent setting, COMB improves throughput by 1.7× and P90 total latency by 1.8×. At equal total concurrency, MAS improves minority-request P90 latency by up to 2.49×. On a 16-core host, simple micro-batching outperforms overlap.
- **Shortcomings:** Request classifications and empirically selected concurrency limits may become unreliable as resource demands change during execution. Scheduling experiments cover fewer workloads than the characterization, and cached pages remove network variability, limiting conclusions about production traffic. Energy profiling does not demonstrate energy savings from the schedulers.

Section 3: Lecture Discussions

The lecture discussion was centered on how AI evolved from standalone models to tool-orchestrating agentic systems. Discussed that CPU processing is a critical bottleneck alongside GPUs. The questions asked during the discussion was about whether proposed scheduling algorithms are uniquely agentic, why GPUs incur high static power draw and the cost-accuracy tradeoffs of scaling smaller models vs larger ones.

Few of the questions that were discussed are: *("a." stands for answers that were told)*

1. Kevin asked if the tool workloads like RAG, database lookups, lexical parsing, etc. are unique to agentic AI or if they existed pre-2022.
    a. They said that the tasks existed long before agents. The fundamental shift is that instead of human programmers or interns writing the code and making queries to the database, the LLMs itself acts as the programmer/intern. It decides which tools to call and decides which environments to generate.
2. In paper 2, the MASK scheduler uses round-robin scheduling for CPU and GPU paths. So would assigning weighted priorities like favoring GPUs over CPUs improve performance ?
    a. The presenters agreed that the weighted/dynamic scheduling based on the tasks would improve performance. The professor added the MASK and COMB are relatively standard pipelining and scheduling concepts. The goal of system design is building a balanced system where no single resource creates a severe bottleneck.
3. Paper 1 shows GPUs idling while waiting for CPU tool execution. But they report that energy consumption is spinning up to 100x. Why is energy consumption so high if the GPU is idling?

a. Even when idle, GPUs draw significant static power, often ¼ ot ⅛ of their tota TDP. Hence, in a long period, the energy accumulates without performing any useful work.

Toward the end, a bunch of questions popped up about the real-world trade-offs and implementations details. Questions like comparing the 8B vs 70B model, where they responded saying big is better for open-ended while smaller is good for structured tasks. Someone also asked a question with energy numbers like if 13.7M joules per query is actually realistic, and Prof. Moshraf said to take academic estimates with a grain of salt unless backed by real industry data center's data.

Ultimately, the discussion highlighted that optimizing agentic AI required treating it as a balanced, hybrid CPU-GPU scheduling challenge rather than a purely GPU-bound task.

Section 4: Proposal for follow-up Work

The two lecture papers each characterize half of agent serving. Kim et al. take the GPU side: parallel tree search gives the best latency-accuracy point, but they caution that it raises memory pressure and may limit multi-tenant scalability, and their tools are cheap to re-run. Raj et al. take the CPU side: stateful tools such as Bash take up to 65% of SWE-Agent latency, CPU and GPU phases are largely disjoint, and oversubscribed cores saturate throughput. Neither covers branching agents with stateful tools under multi-tenant load. The fork mechanism exists: agentfork forks a microVM sandbox and its KV cache as one branch. But it states it is not a scheduler, exposes only fork and kill, and reports results from one A10-class GPU with a 0.6B model, never running a real GPU forward alongside microVMs.

We propose CoFork, a branch scheduler over this primitive, with three policies. Budget: a global cap on running tool branches near 1 to 2 times the core count (COMB), split into a guaranteed minimum per request plus a shared reserve, which is MAS's structure re-keyed from request type to request. Overlap: sibling branches are staggered so some run Bash while others decode a branch-level COMB. Suspension: CoFork adds a suspend/resume pair. Under GPU memory pressure, the lowest-value branches release their KV cache, keep their sandbox, and re-prefill on resume. Prefill is a small share of GPU time in Kim et al., while Bash replay costs CPU-seconds and may not reproduce. Fan-out grows only while the LATS evaluator's best value keeps improving per joule spent. CoFork assumes tool effects stay inside the sandbox.

We would run LATS over a Bash tool on APPS, with unit-test pass rate as the reward, on SGLang and a 64-core server with one 96 GB GPU. Baselines are unmanaged forking, a branch-unaware COMB cap, replay (stock prefix caching, with each branch's sandbox rebuilt by re-running its commands), and sequential search. We measure P50/P90 latency under Poisson arrivals, GPU memory, and accuracy per joule counting CPU energy (via RAPL) and GPU energy (via nvidia-smi), since Kim et al. report GPU energy only. Our hypothesis is that once the baseline oversubscribes cores, CoFork cuts P90 latency by at least 1.5 times against the branch-unaware cap at equal accuracy. The risks are that search values may be too noisy to rank branches, and that cold re-prefill of long coding contexts may cost more than Kim et al.'s short-context numbers suggest.

Section 5: Reflection on AI Usage

- **Shreya:** For Paper 1, AI noted that the energy projections excluded batching and counted only GPU energy, but did not explain the effect of either omission. I prompted it to clarify that batching could lower GPU energy per request, while CPU, network, and cooling would add to total energy. Since these factors pull the estimate in opposite directions, the projection is uncertain. For Paper 2, I asked AI to clarify that COMB's best gains came from a specific experiment and to keep the 16-core result where overlap hurts performance. When a shorter draft left out the MAS evaluation, I added it back.
- **Janani:** For Paper 1 and Paper 2, I used AI primarily to help clarify technical concepts i.e. if I was rightly understanding concepts by running by the summaries I wrote myself, organized my understanding of the papers, and improve the structure and clarity of my summaries especially in terms of grammatical structure and well as reducing content. I independently read and interpreted the papers and used AI for the above mentioned tasks.
- **Bhargav**: I drafted Section 4 myself and used Claude Opus 5.5 to check it against the papers and the agentfork README. It caught overstated claims and the SGLang/vLLM mismatch, but its search missed agentfork until I supplied the URL.